



SLURM: getting a job onto a cluster

How a shared machine decides whose job runs next

Carlos Cotrini · Presented by Ghali

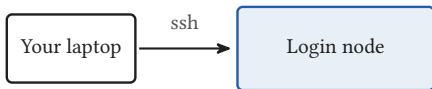


Section 1

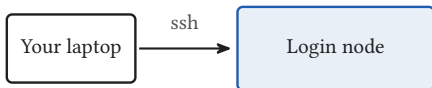
The shared machine

1

The login node is not the cluster



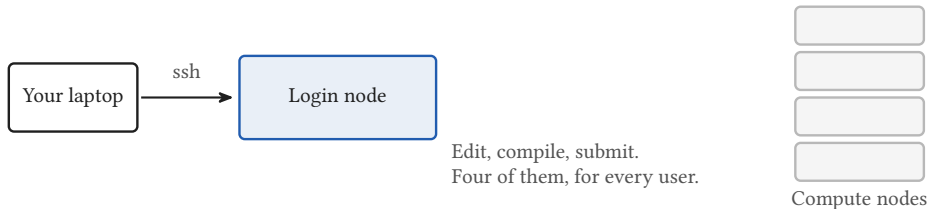
The login node is not the cluster



Edit, compile, submit.
Four of them, for every user.

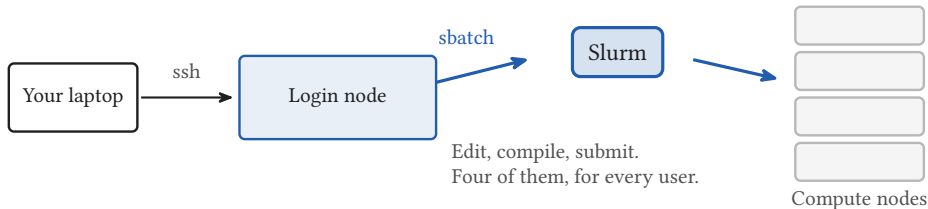
- Not a bigger laptop: a shared machine with an allocation policy

The login node is not the cluster



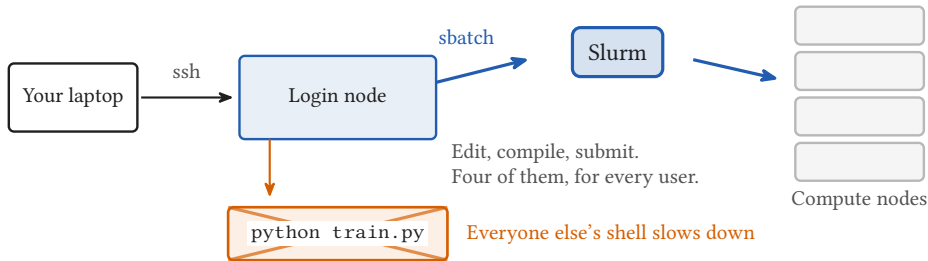
- Not a bigger laptop: a shared machine with an allocation policy

The login node is not the cluster



- Not a bigger laptop: a shared machine with an allocation policy
- Only Slurm puts work on a compute node, and `sbatch` is how you ask

The login node is not the cluster



- Not a bigger laptop: a shared machine with an allocation policy
- Only Slurm puts work on a compute node, and `sbatch` is how you ask
- Running the training on the login node is the classic first mistake

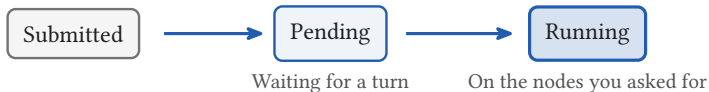
Three partitions, four states

Submitted

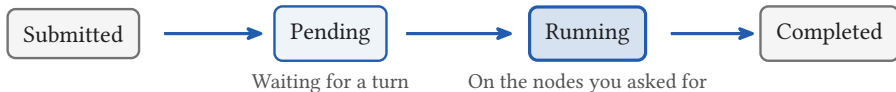
Three partitions, four states



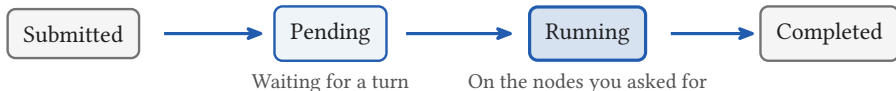
Three partitions, four states



Three partitions, four states



Three partitions, four states



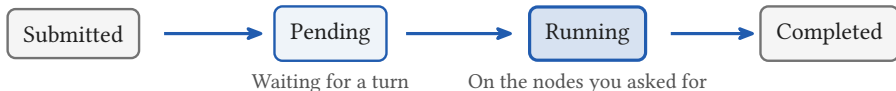
Three partitions on this machine, each with its own limits

debug
32 nodes
1 to 2 nodes per job, 30 min

normal, the default
1266 nodes
No node limit, 24 h

xfer
2 nodes
1 node per job, 24 h

Three partitions, four states



Three partitions on this machine, each with its own limits

debug
32 nodes
1 to 2 nodes per job, 30 min

normal, the default
1266 nodes
No node limit, 24 h

xfer
2 nodes
1 node per job, 24 h

Node sharing is not available on Alps: every job gets whole nodes.

One sbatch script, nine lines

`#!/bin/bash`

The shell that runs it



One sbatch script, nine lines

```
#!/bin/bash
```

```
#SBATCH --account=<project>
```

The shell that runs it

Which budget pays



One sbatch script, nine lines

```
#!/bin/bash  
#SBATCH --account=<project>  
#SBATCH --partition=normal
```

The shell that runs it
Which budget pays
Which pool of nodes



One sbatch script, nine lines

```
#!/bin/bash
#SBATCH --account=<project>
#SBATCH --partition=normal
#SBATCH --nodes=1
```

The shell that runs it
Which budget pays
Which pool of nodes
How many nodes

One node, four GPUs



One sbatch script, nine lines

```
#!/bin/bash
#SBATCH --account=<project>
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
```

The shell that runs it
Which budget pays
Which pool of nodes
How many nodes
One process on the node

One node, four GPUs



One sbatch script, nine lines

```
#!/bin/bash
#SBATCH --account=<project>
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --gpus-per-task=1
```

The shell that runs it

Which budget pays

Which pool of nodes

How many nodes

One process on the node

One GPU for that process

One node, four GPUs



One sbatch script, nine lines

```
#!/bin/bash
#SBATCH --account=<project>
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --gpus-per-task=1
#SBATCH --time=01:00:00
```

The shell that runs it

Which budget pays

Which pool of nodes

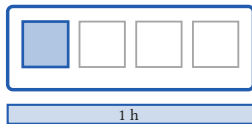
How many nodes

One process on the node

One GPU for that process

Stopped after one hour

One node, four GPUs



One sbatch script, nine lines

```
#!/bin/bash
#SBATCH --account=<project>
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --gpus-per-task=1
#SBATCH --time=01:00:00
#SBATCH --output=logs/%x-%j.log
```

The shell that runs it

Which budget pays

Which pool of nodes

How many nodes

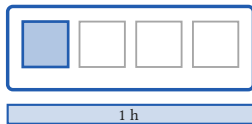
One process on the node

One GPU for that process

Stopped after one hour

Where the output lands

One node, four GPUs



One sbatch script, nine lines

```
#!/bin/bash
#SBATCH --account=<project>
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --gpus-per-task=1
#SBATCH --time=01:00:00
#SBATCH --output=logs/%x-%j.log
srun python train.py
```

The shell that runs it

Which budget pays

Which pool of nodes

How many nodes

One process on the node

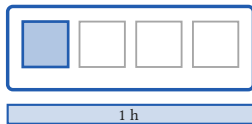
One GPU for that process

Stopped after one hour

Where the output lands

Launch it on the allocation

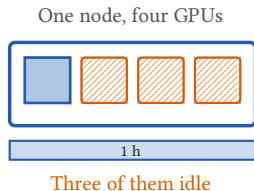
One node, four GPUs



One sbatch script, nine lines

```
#!/bin/bash
#SBATCH --account=<project>
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --gpus-per-task=1
#SBATCH --time=01:00:00
#SBATCH --output=logs/%x-%j.log
srun python train.py
```

The shell that runs it
Which budget pays
Which pool of nodes
How many nodes
One process on the node
One GPU for that process
Stopped after one hour
Where the output lands
Launch it on the allocation



Alps has no node sharing, so this job holds the whole node: four GPUs and 288 cores, one of them used.



Section 2

Why you are waiting

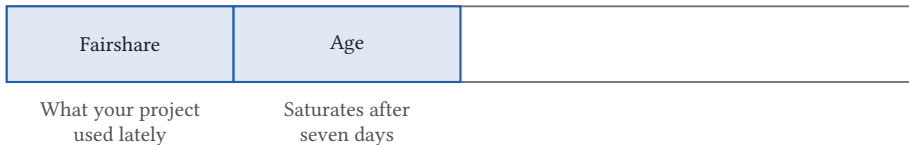
2

Four factors set your place in the queue



What your project
used lately

Four factors set your place in the queue



Four factors set your place in the queue

Fairshare	Age	Job size	
What your project used lately	Saturates after seven days	How many nodes you asked for	

Four factors set your place in the queue

Fairshare	Age	Job size	QOS
What your project used lately	Saturates after seven days	How many nodes you asked for	The service class you asked for

Four factors set your place in the queue

Job priority: one integer, 0 to 4,294,967,295, recomputed every five minutes

Fairshare	Age	Job size	QOS
-----------	-----	----------	-----

What your project
used lately

Saturates after
seven days

How many nodes
you asked for

The service class
you asked for

Four factors set your place in the queue

Job priority: one integer, 0 to 4,294,967,295, recomputed every five minutes

Fairshare	Age	Job size	QOS
-----------	-----	----------	-----

What your project
used lately

Saturates after
seven days

How many nodes
you asked for

The service class
you asked for

And one that is not:

Requested wall time

Nowhere in the formula

Four factors set your place in the queue

Job priority: one integer, 0 to 4,294,967,295, recomputed every five minutes

Fairshare	Age	Job size	QOS
-----------	-----	----------	-----

What your project
used lately

Saturates after
seven days

How many nodes
you asked for

The service class
you asked for

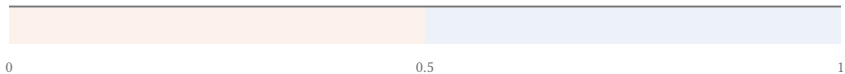
And one that is not:

Requested wall time

Nowhere in the formula

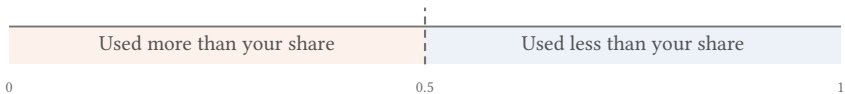
The weight on each factor is a site setting. Run `scontrol show config` to see what this machine actually does.

Fairshare: 0.5 is exactly your share



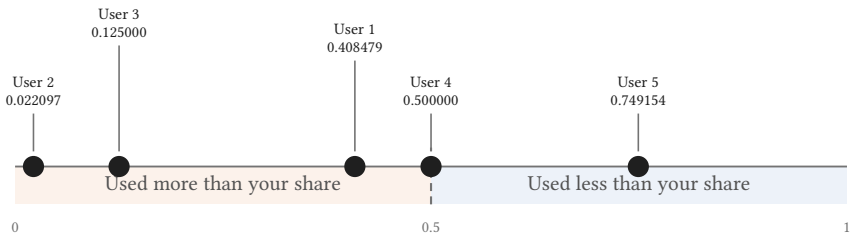
Why you are waiting

Fairshare: 0.5 is exactly your share



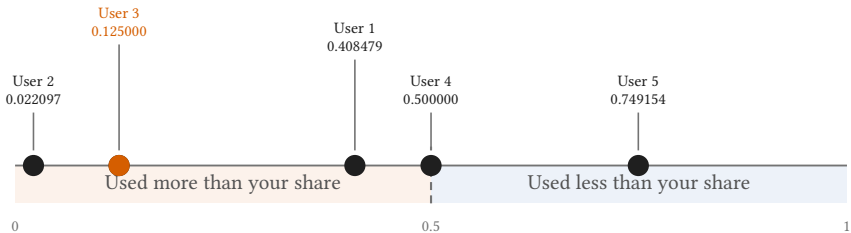
Why you are waiting

Fairshare: 0.5 is exactly your share



Why you are waiting

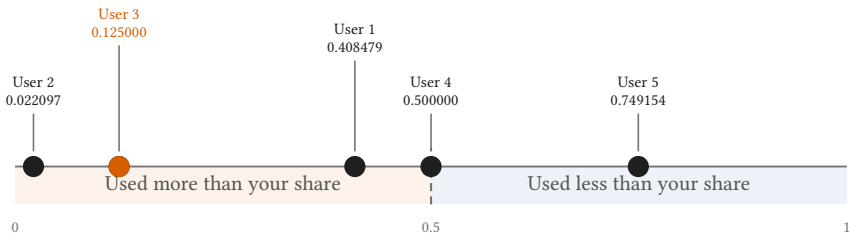
Fairshare: 0.5 is exactly your share



User 3 has run nothing. That number is their account-mates' usage.

Why you are waiting

Fairshare: 0.5 is exactly your share



User 3 has run nothing. That number is their account-mates' usage.

Fairshare is charged to the account, not to you, and usage decays with a seven day half life by default ([PriorityDecayHalfLife](#)).



Section 3

Backfill

3

Four nodes, five jobs

Every job is a rectangle: nodes tall, hours wide



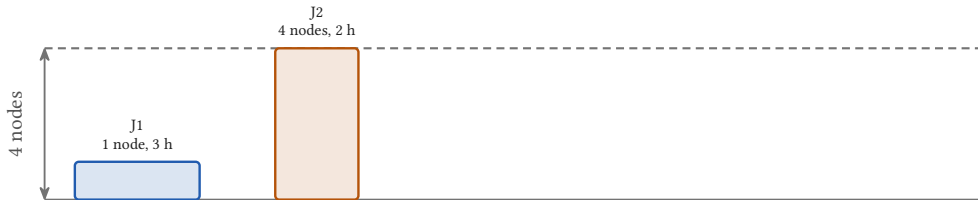
Four nodes, five jobs

Every job is a rectangle: nodes tall, hours wide



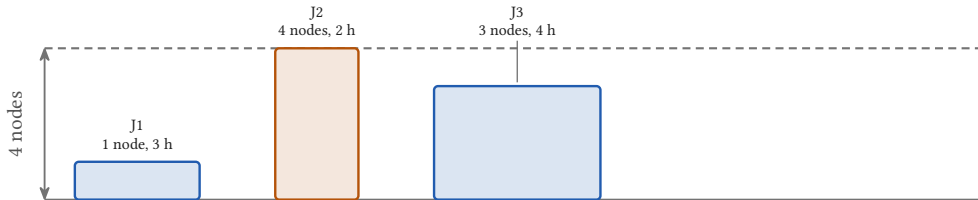
Four nodes, five jobs

Every job is a rectangle: nodes tall, hours wide



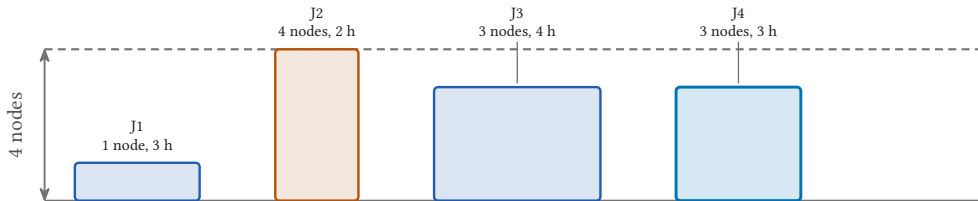
Four nodes, five jobs

Every job is a rectangle: nodes tall, hours wide



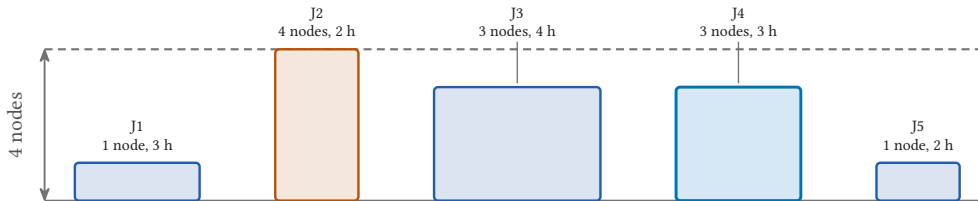
Four nodes, five jobs

Every job is a rectangle: nodes tall, hours wide



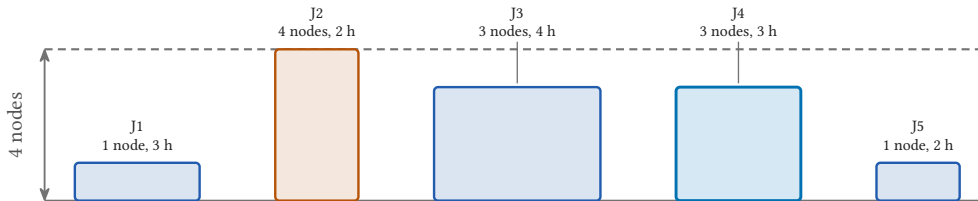
Four nodes, five jobs

Every job is a rectangle: nodes tall, hours wide



Four nodes, five jobs

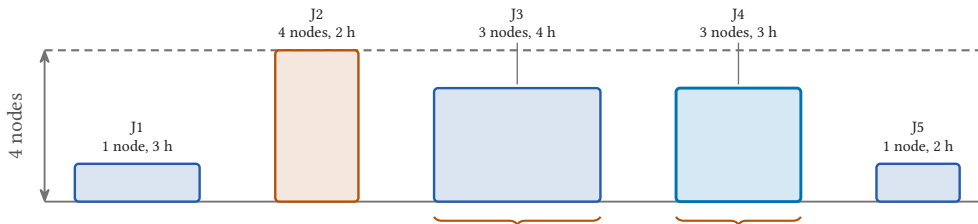
Every job is a rectangle: nodes tall, hours wide



The area is what you are charged: 34 node hours in all

Four nodes, five jobs

Every job is a rectangle: nodes tall, hours wide

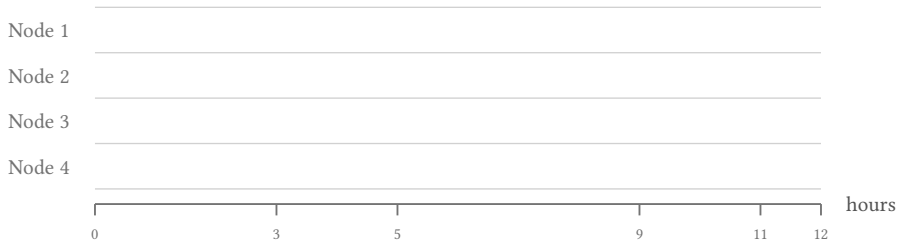


The area is what you are charged: 34 node hours in all

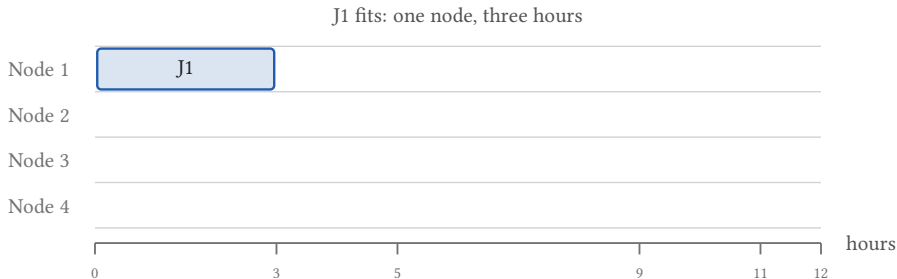
J3 and J4 are the same three nodes. Only the time asked for differs.

All five are submitted at the same moment, and they are listed in priority order.

Strict priority: twelve hours

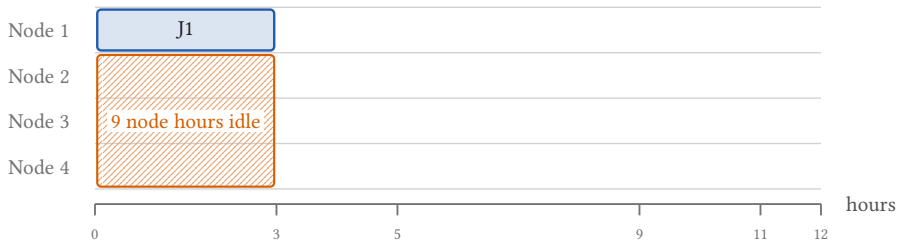


Strict priority: twelve hours

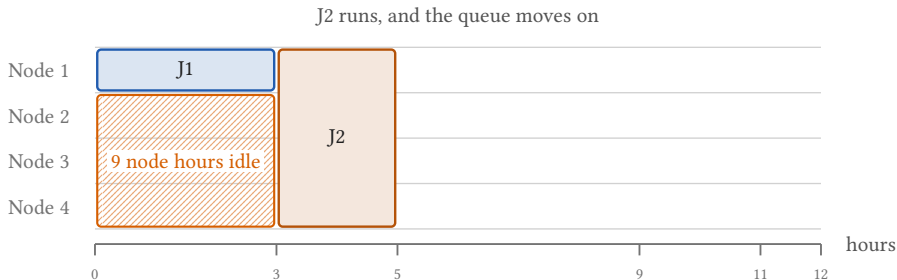


Strict priority: twelve hours

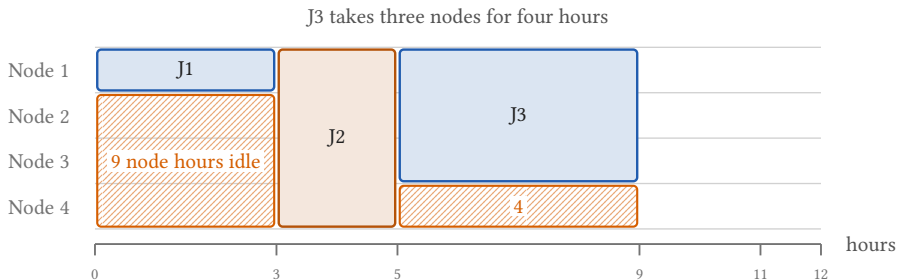
J2 needs all four nodes, so it waits, and so does everything behind it



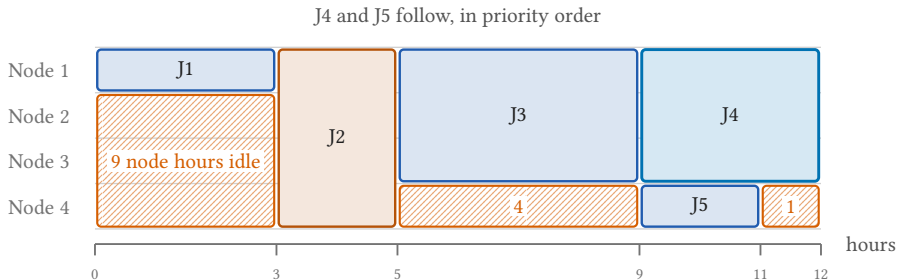
Strict priority: twelve hours



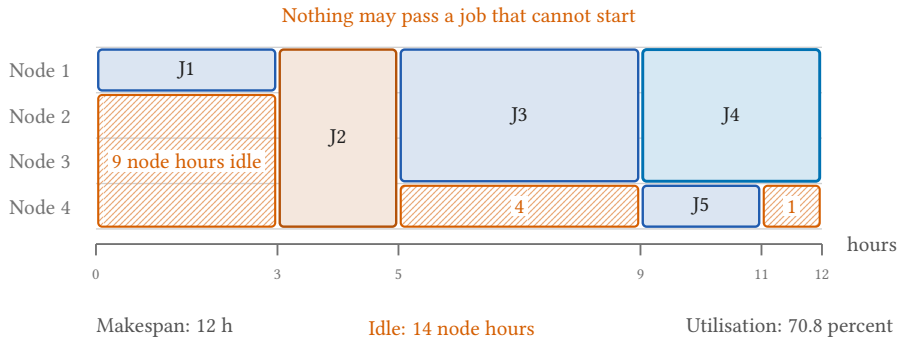
Strict priority: twelve hours



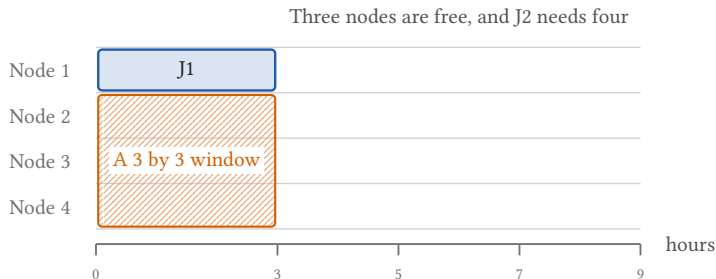
Strict priority: twelve hours



Strict priority: twelve hours

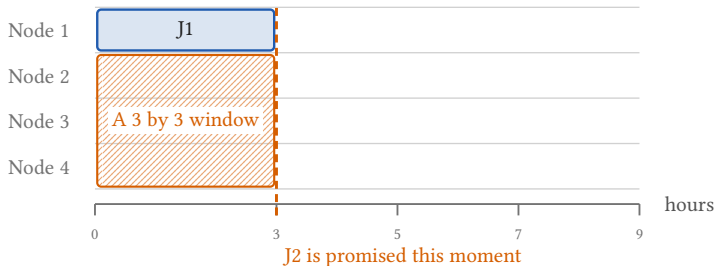


Backfill: the same five jobs, nine hours

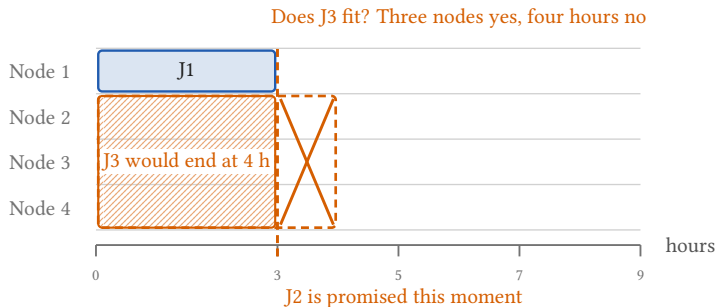


Backfill: the same five jobs, nine hours

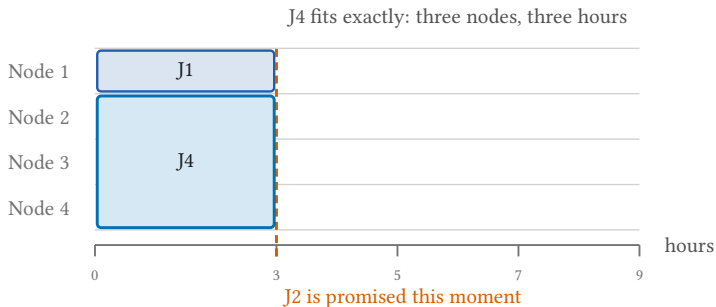
Slurm reserves J2 the moment J1 ends, and will not break that promise



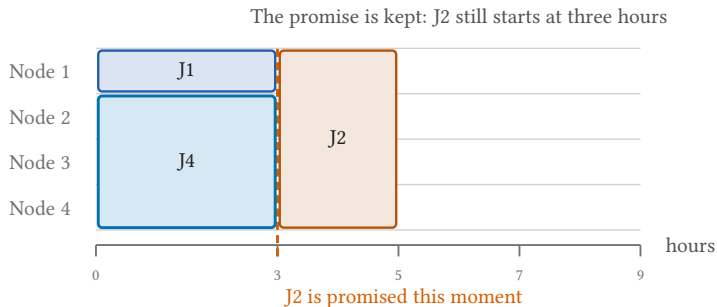
Backfill: the same five jobs, nine hours



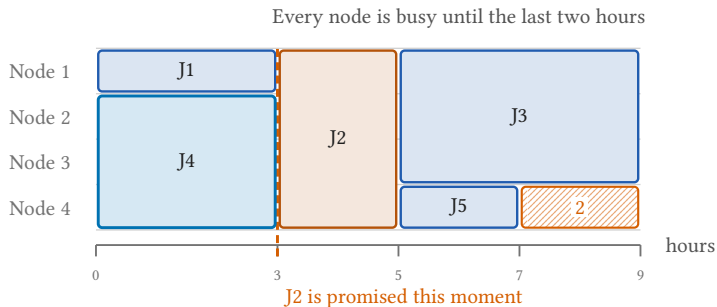
Backfill: the same five jobs, nine hours



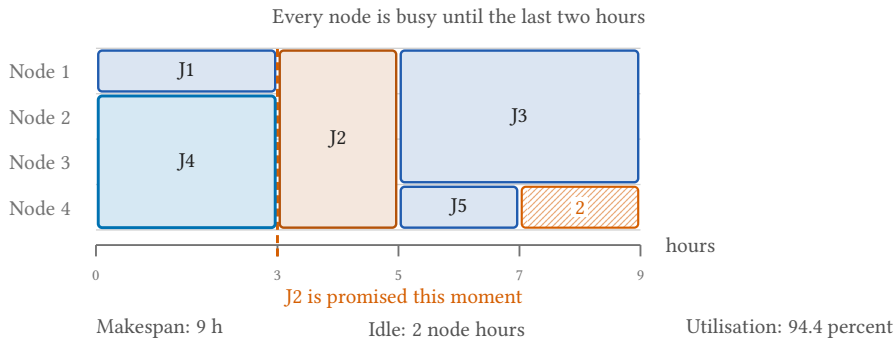
Backfill: the same five jobs, nine hours



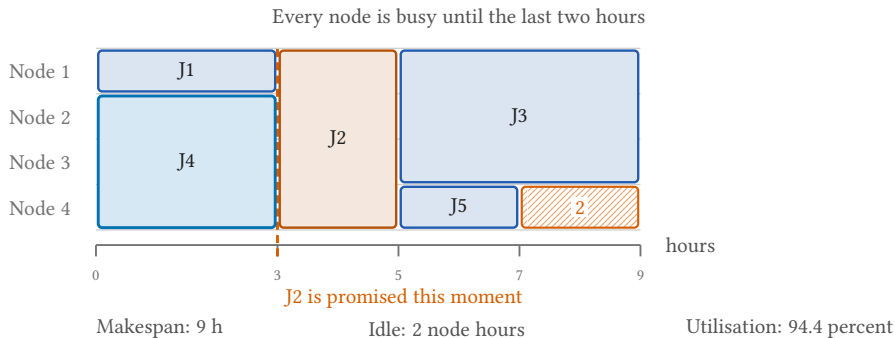
Backfill: the same five jobs, nine hours



Backfill: the same five jobs, nine hours



Backfill: the same five jobs, nine hours

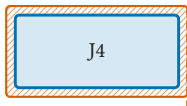


Nodes held for that promise look idle. Slurm calls them **planned**.

A shorter request fits more holes

The window in front of the reservation: three nodes, three hours

J4 asks three hours

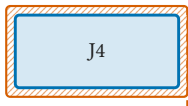


It fits, so it starts now

A shorter request fits more holes

The window in front of the reservation: three nodes, three hours

J4 asks three hours



It fits, so it starts now

J3 asks four hours

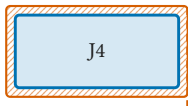


It would end after the promise

A shorter request fits more holes

The window in front of the reservation: three nodes, three hours

J4 asks three hours



It fits, so it starts now

J3 asks four hours



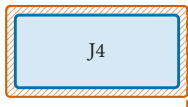
It would end after the promise

- J3 is ahead of J4 in the queue the whole time, and starts five hours later

A shorter request fits more holes

The window in front of the reservation: three nodes, three hours

J4 asks three hours



It fits, so it starts now

J3 asks four hours



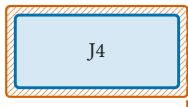
It would end after the promise

- J3 is ahead of J4 in the queue the whole time, and starts five hours later
- A shorter request does not promote you: it makes you [fit](#)

A shorter request fits more holes

The window in front of the reservation: three nodes, three hours

J4 asks three hours



It fits, so it starts now

J3 asks four hours



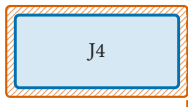
It would end after the promise

- J3 is ahead of J4 in the queue the whole time, and starts five hours later
- A shorter request does not promote you: it makes you [fit](#)
- Had J3 asked for the three hours it needs, it would take the window

A shorter request fits more holes

The window in front of the reservation: three nodes, three hours

J4 asks three hours



It fits, so it starts now

J3 asks four hours



It would end after the promise

- J3 is ahead of J4 in the queue the whole time, and starts five hours later
- A shorter request does not promote you: it makes you [fit](#)
- Had J3 asked for the three hours it needs, it would take the window
- Ask too little and Slurm stops your job at the limit you gave



Section 4

What wastes an allocation

4

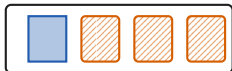
Four ways to waste an allocation



One GPU, a whole node

Four GPUs billed, one used

Four ways to waste an allocation



One GPU, a whole node

Four GPUs billed, one used

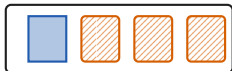


Wall time far above what you need

Costs queue position only



Four ways to waste an allocation



One GPU, a whole node

Four GPUs billed, one used



Wall time far above what you need

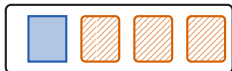
Costs queue position only



A thousand separate submissions

Use one `--array` job

Four ways to waste an allocation



One GPU, a whole node

Four GPUs billed, one used



Wall time far above what you need

Costs queue position only



A thousand separate submissions

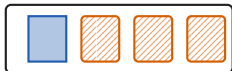
Use one `--array` job



Training on the login node

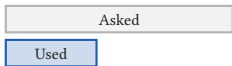
Four of them, for every user

Four ways to waste an allocation



One GPU, a whole node

Four GPUs billed, one used



Wall time far above what you need

Costs queue position only



A thousand separate submissions

Use one `--array` job



Training on the login node

Four of them, for every user

Nodes and GPUs are charged whether you use them. Wall time is not: Slurm charges the time your job actually ran.

Four commands, and what each one shows

1

sinfo



Which partitions exist, and how many nodes are free

Four commands, and what each one shows

- 1 `sinfo` → Which partitions exist, and how many nodes are free
- 2 `sprio -l` → Your priority, broken into its factors

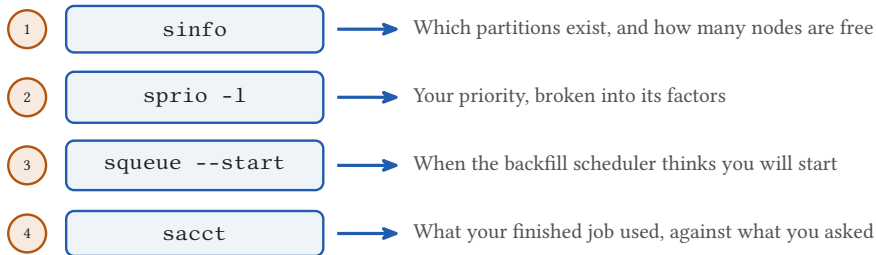
Four commands, and what each one shows

- 1 `sinfo` → Which partitions exist, and how many nodes are free
- 2 `sprio -l` → Your priority, broken into its factors
- 3 `squeue --start` → When the backfill scheduler thinks you will start

Four commands, and what each one shows

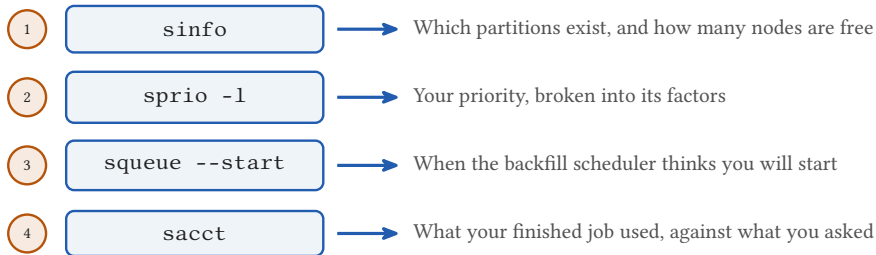
- 1 `sinfo` → Which partitions exist, and how many nodes are free
- 2 `sprio -l` → Your priority, broken into its factors
- 3 `squeue --start` → When the backfill scheduler thinks you will start
- 4 `sacct` → What your finished job used, against what you asked

Four commands, and what each one shows



- The circled number is the section of this deck each command makes visible

Four commands, and what each one shows



- The circled number is the section of this deck each command makes visible
- At 14:00 you write one of these scripts and put it into the queue